

Transformers & Attention Mechanism (Part 3)

Transformer Decoder

Definition

The **Transformer Decoder** is responsible for generating the output sequence one token (word) at a time.

Unlike the Encoder, which only understands the input, the Decoder generates meaningful output by using:

- Previous generated words
- Encoder outputs
- Attention Mechanisms

Each Decoder Layer contains three main components:

1. Masked Multi-Head Self-Attention
2. Encoder–Decoder Attention (Cross Attention)

3.Feed Forward Neural Network (FFN)

Training Phase vs Prediction Phase

The Decoder behaves differently during training and prediction.

Training Phase (Teacher Forcing)

During training, the Decoder already knows the correct previous word.

Instead of using its own prediction, it is given the actual previous word as input.

This technique is called **Teacher Forcing**.

Example:

Input:

Start Token

↓

|



Love



AI

At every step, the correct previous word is provided.

This allows the model to learn faster and reduces error propagation during training.

Prediction Phase (Inference)

During prediction, the Decoder does **not** know the correct output.

It must generate each word one by one.

Workflow:

Start Token



Predict First Word



Use First Word to Predict Second Word



Use Second Word to Predict Third Word



Continue Until End Token

The Decoder depends entirely on its previous prediction.

Auto-Regressive Generation

Definition

A Transformer Decoder generates text in an **Auto-Regressive** manner.

This means every newly generated word becomes the input for predicting the next word.

Example:

Start



I



Love



Machine



Learning

Each prediction depends on all previously generated words.

Large Language Models such as GPT follow this Auto-Regressive generation process.

Non-Auto-Regressive Generation

Definition

In a **Non-Auto-Regressive** model, all output words are generated simultaneously rather than one by one.

Advantages:

- Faster inference
- Parallel prediction

Disadvantages:

- Lower accuracy
- Weaker contextual understanding

Modern Large Language Models mostly prefer **Auto-Regressive decoding** because it produces more coherent text.

Masked Multi-Head Self-Attention

Definition

The first layer of the Decoder is called **Masked Multi-Head Self-Attention**.

It works similarly to the Encoder's Self-Attention, but with one important difference:

The Decoder is **not allowed to look at future words**.

It can only attend to:

- Current word
 - Previous words
-

Why Masking?

Suppose the sentence is:

I Love Deep Learning

While predicting **Love**, the model must **not** see:

Deep

Learning

Otherwise, the model would cheat during training.

Therefore, future positions are hidden using a

Mask.

This ensures the Decoder learns to generate text one word at a time.

Masking Example

Sentence

I Love Deep Learning

Attention Matrix

I Love Deep Learning

I	✓	X	X	X
---	---	---	---	---

Love	✓	✓	X	X
------	---	---	---	---

Deep	✓	✓	✓	X
------	---	---	---	---

Learn	✓	✓	✓	✓
-------	---	---	---	---

Every word can only attend to itself and previous words.

Future words remain hidden.

Cross Attention (Encoder–Decoder Attention)

Definition

After Masked Self-Attention, the Decoder performs **Cross Attention**.

Here, the Decoder attends to the Encoder outputs.

This helps the Decoder focus on the relevant parts of the input sentence while generating each output word.

Why Cross Attention?

Example

Input:

English

"I Love Pakistan"

Output:

French

"J'aime le Pakistan"

When generating each French word, the Decoder needs information from the Encoder.

Cross Attention allows this communication.

Query, Key and Value in Cross Attention

Unlike Self-Attention:

Query (Q)



Comes from Decoder

Key (K)



Comes from Encoder

Value (V)



Comes from Encoder

The Decoder asks:

"Which parts of the input sentence are most important for generating the next word?"

Feed Forward Neural Network (Decoder)

After Cross Attention, the output passes through another **Feed Forward Neural Network (FFN)**.

Its purpose is the same as in the Encoder.

It learns deeper representations for every generated word.

Structure:

Linear



Activation Function



Linear

Again,

Residual Connections and Layer Normalization are applied around every major sub-layer.

Decoder Layer Workflow

Complete Decoder Layer:

Masked Multi-Head Self-Attention



Add & Layer Normalization



Cross Attention



Add & Layer Normalization



Feed Forward Neural Network



Add & Layer Normalization



Next Decoder Layer

Complete Transformer Architecture

The complete Transformer consists of two major blocks:

Encoder



Decoder

Encoder Workflow

Input Sentence



Word Embeddings



Positional Encoding



Multi-Head Self-Attention



Add & Layer Normalization



Feed Forward Neural Network



Add & Layer Normalization



Repeat Multiple Encoder Layers



Encoder Output

Decoder Workflow

Previous Output Words



Word Embeddings



Positional Encoding



Masked Multi-Head Self-Attention



Add & Layer Normalization



Cross Attention (Using Encoder Output)



Add & Layer Normalization



Feed Forward Neural Network



Add & Layer Normalization



Repeat Multiple Decoder Layers



Linear Layer



Softmax



Predicted Next Word

Complete Transformer Pipeline

Input Sentence



Embedding



Positional Encoding



Encoder Stack



Encoder Output



Decoder Input (Start Token / Previous Predictions)



Masked Multi-Head Self-Attention



Cross Attention



Feed Forward Neural Network



Linear Layer



Softmax



Next Word Prediction



Repeat Until End Token

Advantages of Transformers

- Parallel processing (faster than RNNs)
- Captures long-range dependencies
- Handles long sequences effectively
- Better contextual understanding
- Highly scalable
- State-of-the-art performance in NLP
- Forms the foundation of modern Large Language Models (LLMs)

Limitations of Transformers

- Require large amounts of training data
- Computationally expensive
- High memory consumption
- Attention complexity increases with sequence length ($O(n^2)$)

- Expensive to train from scratch

Applications of Transformers

Transformers are used in a wide range of AI applications:

- Machine Translation
 - Question Answering
 - Chatbots
 - Text Summarization
 - Text Generation
 - Sentiment Analysis
 - Code Generation
 - Speech Recognition
 - Image Captioning
 - Vision Transformers (ViT)
 - Multimodal AI (Text + Image + Audio)
-

From ANN to Transformers (Journey)

ANN



Learns fixed-size structured data.



CNN



Learns spatial information from images.



RNN



Learns sequential data but suffers from Vanishing Gradient.



LSTM



Introduces memory cells to capture long-term dependencies.



GRU



Simplifies LSTM while maintaining performance.



Encoder–Decoder



Learns Sequence-to-Sequence tasks.



Attention Mechanism



Allows the Decoder to focus on important input words.



Transformers



Remove recurrence completely and rely entirely on Self-Attention.



Large Language Models (LLMs)

Examples:

- GPT
- BERT
- T5
- LLaMA
- Gemini

Final Summary

Artificial Neural Networks (ANN) are suitable for fixed-size structured data.

Convolutional Neural Networks (CNN) specialize in extracting spatial features from images.

Recurrent Neural Networks (RNN) process sequential data but struggle with long-term dependencies due to the Vanishing Gradient Problem.

LSTM and GRU improve RNNs by introducing gating mechanisms that help preserve important information over longer sequences.

Encoder–Decoder architectures make Sequence-to-Sequence tasks such as machine translation possible by first understanding the input and then generating the output.

The Attention Mechanism overcomes the limitation of a single fixed Context Vector by allowing the model to focus on the most relevant parts of the input.

Finally, **Transformers** eliminate recurrence altogether and use **Self-Attention** to process all words in parallel, enabling faster training, better context understanding, and superior performance.

This architecture has become the foundation of modern Generative AI and today's Large Language Models.